

Compilation Techniques: Laboratory

Types

Giuseppe Lomurno, University of Pisa

March 4th, 2026

The objective:

- Prove properties about the behaviour of the program without executing it
- Decidability, even if it means approximation (Rice theorem)

Which properties?

- Wrong logic
- Run-time errors
- Vulnerabilities
- Performance issues

Recall

Deadlocks in MiniFun

```
letfun f x = true + y in f 3
```

Two identifiable problems:

- the variable y is undefined
- the addition is not defined between booleans and integer values

The program has no valid semantics

The most common static analysis technique

A type abstract over a class of terms that share some commonalities

- Run-time representation (think of C `int64_t` vs `char`)
- Interface operations (think of Java `List` or Haskell `Monad`)
- Theorems (think of type-based theorem provers like Agda or Rocq)

For MiniFun: no invalid operations at run-time which implies no deadlocks

Project

Idea

- ① We identify types for terms, $t : \tau$
- ② We build a deduction system to infer the type
- ③ We check for inconsistencies which imply contradictions
- ④ We devise a procedure to implement the deduction system

Inconsistencies

Example

```
letfun f x = x and true in  
  f 5
```

`x and true` is a valid term, we expect `x` to be a boolean

`f 5` is a valid term, we expect `x` to be an integer

Boolean and integers are inconsistent, e.g. operations expect one or the other but not both

Idea: the deduction system computes and propagates constraints

Type analysis is still an approximation

Some terms will be deemed inconsistent even if intuitively are correct

```
let fun f x = f (x - 1) in  
  fun x => (x + f x) and true
```

There will always be cases like this, no matter how hard you try!

As always, tradeoffs are important: more precision translates to harder problems

A Simple Type System

We will denote types with $\tau, \tau', \dots$, and the set of all types **Type**

$$\tau ::= \text{int} \mid \text{bool} \mid \tau \rightarrow \tau$$

The syntax $\tau \rightarrow \tau'$ represents functions with an argument τ that produces a term τ'

We assume different types are incompatible: no subtyping or implicit casting

$t : \tau$ and $t : \tau'$ with $\tau \neq \tau'$ is always a contradiction

If contradiction raise a type checking failure

Context

Problem

What is the typing?

```
fun x => x + y
```

We expect $\text{int} \rightarrow \text{int}$, but what if y is a `bool`?

Context

Problem

What is the typing?

```
fun x => x + y
```

We expect $\text{int} \rightarrow \text{int}$, but what if y is a `bool`?

Typing is contextual:

```
let y = 1 in  
  fun x => x + y
```

The typing is $\text{int} \rightarrow \text{int}$

Context

Problem

What is the typing?

```
fun x => x + y
```

We expect $\text{int} \rightarrow \text{int}$, but what if y is a `bool`?

Typing is contextual:

```
let y = true in  
  fun x => x + y
```

There is an inconsistency, raise an error

Context

Problem

What is the typing?

```
fun x => x + y
```

We expect $\text{int} \rightarrow \text{int}$, but what if y is a `bool`?

Typing is contextual:

```
fun x => x + y
```

y is undefined, raise an error

Context

Definition

We need to store contextual information about variables

Formally, a typing context Γ is a partial function

$$\Gamma : X \mapsto \mathbf{Type}$$

```
let a = 1 in  
let b = true in  
  fun x => x < a and b
```

When typechecking the function we expect $\Gamma = \{(a, \text{int}), (b, \text{bool})\}$

A typing judgement is a sequent of the form $\Gamma \vdash t : \tau$

- Read it as “in the context Γ , the term t has type τ ”
- We deduct τ from the types of the subterms of t and the types in the context
- The type of t is needed for terms t is a subterm of or when its name is used
- We are interested in whether t has a type or not
- We say a term t as type τ when $\emptyset \vdash t : \tau$

Literals are trivial

Num

$$\frac{}{\Gamma \vdash n : \text{int}}$$

BoolT

$$\frac{}{\Gamma \vdash \text{true} : \text{bool}}$$

BoolF

$$\frac{}{\Gamma \vdash \text{false} : \text{bool}}$$

Variables are typed according to the context

Var

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$

Operations are trivial (the other are left for you to guess)

$$\begin{array}{c} \text{Plus} \\ \Gamma \vdash t_1 : \text{int} \quad \Gamma \vdash t_2 : \text{int} \\ \hline \Gamma \vdash t_1 + t_2 : \text{int} \end{array}$$

Conditionals require both branch to have the same type, why?

$$\begin{array}{c} \text{ITE} \\ \Gamma \vdash t_1 : \text{bool} \quad \Gamma \vdash t_2 : \tau \quad \Gamma \vdash t_3 : \tau \\ \hline \Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : \tau \end{array}$$

Functions bind variables, we must update the context

$$\text{Fun} \quad \frac{\Gamma[x \mapsto \tau] \vdash t : \tau'}{\Gamma \vdash \text{fun } x \Rightarrow t : \tau \rightarrow \tau'}$$

$$\text{FunApp} \quad \frac{\Gamma \vdash t_1 : \tau \rightarrow \tau' \quad \Gamma \vdash t_2 : \tau}{\Gamma \vdash t_1 t_2 : \tau'}$$

Note how the typing rules are similar to the semantics rules

Remember: types can abstract over run-time behaviour

Lets are similar to functions

$$\text{Let} \quad \frac{\Gamma \vdash t_1 : \tau \quad \Gamma[x \mapsto \tau] \vdash t_2 : \tau'}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : \tau'}$$

In recursive lets we must also remember the type of the recursive function

$$\text{LetFun} \quad \frac{\Gamma[f \mapsto \tau \rightarrow \tau'][x \mapsto \tau] \vdash t_1 : \tau' \quad \Gamma[f \mapsto \tau \rightarrow \tau'] \vdash t_2 : \tau''}{\Gamma \vdash \text{letfun } f \ x = t_1 \text{ in } t_2 : \tau''}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{}{\emptyset \vdash (\text{fun } x \Rightarrow x) \text{ } 3 : \tau'}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{\overline{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau'} \quad \overline{\emptyset \vdash 3 : \tau}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{ App}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{\frac{\overline{\emptyset[x \mapsto \tau] \vdash x : \tau'}}{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau'} \text{ Fun} \quad \overline{\emptyset \vdash 3 : \tau}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{ App}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{\frac{\frac{\emptyset[x \mapsto \tau](x) = \tau}{\emptyset[x \mapsto \tau] \vdash x : \tau'} \text{ Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau'} \text{ Fun} \quad \frac{}{\emptyset \vdash 3 : \tau}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{ App}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{\frac{\frac{\emptyset[x \mapsto \tau](x) = \tau}{\emptyset[x \mapsto \tau] \vdash x : \tau} \text{ Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau} \text{ Fun} \quad \frac{}{\emptyset \vdash 3 : \tau}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau} \text{ App}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{\frac{\frac{\emptyset[x \mapsto \tau](x) = \tau}{\emptyset[x \mapsto \tau] \vdash x : \tau} \text{ Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau} \text{ Fun} \quad \frac{}{\emptyset \vdash 3 : \text{int}} \text{ Num}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau} \text{ App}$$

Propagation

Example

In the implementation you have to propagate constraining information

$$\frac{\frac{\frac{\emptyset[x \mapsto \text{int}](x) = \text{int}}{\emptyset[x \mapsto \text{int}] \vdash x : \text{int}} \text{Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \text{int} \rightarrow \text{int}} \text{Fun} \quad \frac{}{\emptyset \vdash 3 : \text{int}} \text{Num}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \text{int}} \text{App}$$

Propagation

Example

The order matters!

$$\frac{}{\emptyset \vdash (\text{fun } x \Rightarrow x) \text{ } 3 : \tau'}$$

Propagation

Example

The order matters!

$$\frac{\overline{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau'} \quad \overline{\emptyset \vdash 3 : \tau}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{ App}$$

Propagation

Example

The order matters!

$$\frac{\overline{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau'}}{\overline{\emptyset \vdash (\text{fun } x \Rightarrow x) \ 3 : \tau'}} \quad \frac{\overline{\emptyset \vdash 3 : \text{int}}}{\text{Num}} \quad \text{App}$$

Propagation

Example

The order matters!

$$\frac{\frac{}{\emptyset \vdash \text{fun } x \Rightarrow x : \text{int} \rightarrow \tau'}}{\quad} \quad \frac{}{\emptyset \vdash 3 : \text{int}} \text{Num}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{App}$$

Propagation

Example

The order matters!

$$\frac{\frac{\overline{\emptyset[x \mapsto \text{int}] \vdash x : \tau'}}{\emptyset \vdash \text{fun } x \Rightarrow x : \text{int} \rightarrow \tau'} \text{ Fun} \quad \frac{}{\emptyset \vdash 3 : \text{int}} \text{ Num}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{ App}$$

Propagation

Example

The order matters!

$$\frac{\frac{\frac{\emptyset[x \mapsto \text{int}](x) = \text{int}}{\emptyset[x \mapsto \text{int}] \vdash x : \tau'} \text{Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \text{int} \rightarrow \tau'} \text{Fun} \quad \frac{}{\emptyset \vdash 3 : \text{int}} \text{Num}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \tau'} \text{App}$$

Propagation

Example

The order matters!

$$\frac{\frac{\frac{\emptyset[x \mapsto \text{int}](x) = \text{int}}{\emptyset[x \mapsto \text{int}] \vdash x : \text{int}} \text{Var} \quad \frac{}{\emptyset \vdash 3 : \text{int}} \text{Num}}{\emptyset \vdash (\text{fun } x \Rightarrow x) 3 : \text{int}} \text{App}$$

What happens when constraints are not enough?

$$\frac{\frac{\emptyset[x \mapsto \tau](x) = \tau}{\emptyset[x \mapsto \tau] \vdash x : \tau} \text{Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau} \text{Fun}$$

What happens when constraints are not enough?

$\tau = \text{int}$? consistent

$$\frac{\frac{\emptyset[x \mapsto \text{int}](x) = \text{int}}{\emptyset[x \mapsto \text{int}] \vdash x : \text{int}} \text{Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \text{int} \rightarrow \text{int}} \text{Fun}$$

Propagation

Constraints

What happens when constraints are not enough?

$\tau = \text{bool} \rightarrow \text{bool}$? consistent

$$\frac{\frac{\emptyset[x \mapsto \text{bool} \rightarrow \text{bool}](x) = \text{bool} \rightarrow \text{bool}}{\emptyset[x \mapsto \text{bool} \rightarrow \text{bool}] \vdash x : \text{bool} \rightarrow \text{bool}} \text{Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : (\text{bool} \rightarrow \text{bool}) \rightarrow (\text{bool} \rightarrow \text{bool})} \text{Fun}$$

Propagation

Constraints

All solutions are consistent, this is not a problem in theory

If there is a solution then the term is well-typed

We may want the most general guess, or a specific guess, e.g. to guide code generation

What solutions?

- Use a heuristic, no input required, but no control
- Add type annotation to manually guide the typechecking (next)
- Use a more expressive type system (later)

Type Annotations

Ask the programmer

```
<t> ::= <int> | true | false | <var> | fun <var> : <type> => <t> |  
      <t> <t> | <t> <binop> <t> | ~ <t> |  
      if <t> then <t> else <t> | let <var> = <t> in <t> |  
      letfun <var> <var> : <type> = <t> in <t>  
  
<binop> ::= + | - | * | && | <  
<type> ::= int | bool | <type> -> <type>
```

Something to pay attention to:

- The type of `letfun` must be a function, i.e. $\tau \rightarrow \tau'$ where τ is the type of the argument and τ' of the resulting term
- Functional types are right associative, i.e. $\tau_1 \rightarrow \tau_2 \rightarrow \tau_3 = \tau_1 \rightarrow (\tau_2 \rightarrow \tau_3)$
- Why the `let` does not require an annotation?

Project

Fragment 3

- 1 Complete the type system of MiniFun with rules for handling the annotations
- 2 Write the rules in the report (just the one you devised)
- 3 Extend the syntax of MiniFun with annotations
- 4 Define data types for types and environments
- 5 Translate the deduction system into a typechecking function that either produces the type of a term or errors out
- 6 Bonus points if the error explains why it fails to typecheck

A Polymorphic Type System

Generic functions

Example

Recall the example without a unique type

$$\frac{\frac{\emptyset[x \mapsto \tau](x) = \tau}{\emptyset[x \mapsto \tau] \vdash x : \tau} \text{ Var}}{\emptyset \vdash \text{fun } x \Rightarrow x : \tau \rightarrow \tau} \text{ Fun}$$

The identity function behaves the same regardless of the specific type τ

Generic functions

Typing

How do we type `id`?

```
let id = fun x => x in
let y = id 2 in
  id (y < 3)
```

Either `int` or `bool`, with or without annotations, leads to an error

We can define a function for each distinct usage

Can be done automatically according to usage, e.g. monomorphization in C++ or Rust

Generic functions

Typing

How do we type `id`?

```
let id = fun x => x in  
let y = id 2 in  
  id (y < 3)
```

Either `int` or `bool`, with or without annotations, leads to an error

We can define a function for each distinct usage

Can be done automatically according to usage, e.g. monomorphization in C++ or Rust

We can quantify universally over types, and instantiate when needed

$$\text{id} : \forall \tau. \tau \rightarrow \tau$$

Parametric Polymorphism

Objective

We will define a polymorphic type system, a variant of the Hindley-Milner type system

Objective: return the most general type without type annotations

- We introduce type variables: $\text{const} : a \rightarrow b \rightarrow a$
- We implicitly define an order on types:
 $a \rightarrow b \rightarrow a \sqsubseteq \text{int} \rightarrow b \rightarrow \text{int} \sqsubseteq \text{int} \rightarrow \text{bool} \rightarrow \text{int}$
Read $a \sqsubseteq b$ as "type a is more general than type b "
- When in need, we instantiate the most general type, principal type, with the required parameters

Parametric Polymorphism

Types

We differentiate types in two classes: monotypes and polytypes

Monotypes: $\tau ::= \alpha \mid \text{int} \mid \text{bool} \mid \tau \rightarrow \tau$

Polytypes: $\sigma ::= \tau \mid \forall \alpha. \sigma$

Note 1: Quantifiers can only appear at the top-level, e.g. $\forall \alpha. \alpha \rightarrow \forall \alpha. \alpha$ is not allowed

Note 2: Two polytypes are equal up-to reordering of the quantifiers and α -renaming

Typing contexts Γ are function from variables to polytypes

$$\Gamma : X \multimap \mathbf{Polytype}$$

We also update typing judgements to polytypes: $\Gamma \vdash x : \sigma$

Substitution

Definition

To obtain monotypes, i.e. without variables, we need substitutions

Formally, a substitution is a function from variables to monotypes

$$S : X \rightarrow \mathbf{Monotype}$$

We will write a substitution as $S = \{\alpha \mapsto \tau_1, \beta \mapsto \tau_2\}$, where it is assumed that the substitution acts as the identity for unspecified variables

Applying a substitution, $S\tau$ or $S\sigma$, means to simultaneously substitute all occurrences of a variable with its mapped type

Substitution

Examples

Take $S = \{\alpha \mapsto \text{int}, \gamma \mapsto \text{bool}\}$

Substitution in ground terms is trivial

$$S(\alpha \rightarrow \text{bool} \rightarrow \beta \rightarrow \alpha) = \text{int} \rightarrow \text{bool} \rightarrow \beta \rightarrow \text{int}$$

For polytypes treat unbound variables as constants

$$S(\forall \alpha. \forall \beta. \alpha \rightarrow \beta \rightarrow \gamma) = \forall \beta. \text{int} \rightarrow \beta \rightarrow \gamma$$

For contexts, apply the substitution to each pairing

$$S\Gamma[\alpha \mapsto \sigma] = (S\Gamma)[\alpha \mapsto S\sigma]$$

Algorithm W

A syntax-driven algorithmic procedure to decidably infer the types

The input is a term t and a context Γ and produces a type τ and a substitution S , or fails

We write $\Gamma \vdash t : \tau, S$

- 1 It traverses the syntax recursively
- 2 When it reaches an unknown type, it creates a new corresponding type variable
- 3 When a constraint specialized a variable, it gets registered in the output substitution

Important: The algorithm assumes we have a procedure to generate fresh type variables

Algorithm W

Step-by-Step

Lets start with the more constraining cases

Num

$$\overline{\Gamma \vdash n : \text{int}, \emptyset}$$

BoolT

$$\overline{\Gamma \vdash \text{true} : \text{bool}, \emptyset}$$

BoolF

$$\overline{\Gamma \vdash \text{false} : \text{bool}, \emptyset}$$

Algorithm W

Step-by-Step

$$\frac{\text{Var} \quad \Gamma(x) = \sigma \quad \tau = \text{inst}(\sigma)}{\Gamma \vdash x : \tau, \emptyset}$$

We lookup in the context, however we need to convert the polytype σ to a monotype
Fail if the variable is not in the context

$\text{inst}(\sigma)$ replaces all bound variables in σ with fresh type variables

E.g. $\text{inst}(\forall\alpha. \forall\beta. \alpha \rightarrow \beta) = a \rightarrow b$ with a and b fresh

Algorithm W

Step-by-Step

$$\text{Fun} \quad \frac{\tau \text{ fresh} \quad \Gamma[x \mapsto \tau] \vdash t : \tau', S}{\Gamma \vdash \text{fun } x \Rightarrow t : S\tau \rightarrow \tau', S}$$

- 1 Introduce a fresh variable for the argument type, which is still unknown

Algorithm W

Step-by-Step

$$\text{Fun} \quad \frac{\tau \text{ fresh} \quad \Gamma[x \mapsto \tau] \vdash t : \tau', S}{\Gamma \vdash \text{fun } x \Rightarrow t : S\tau \rightarrow \tau', S}$$

- 1 Introduce a fresh variable for the argument type, which is still unknown
- 2 Check the body of the function with the argument type added to the context

Algorithm W

Step-by-Step

$$\text{Fun} \quad \frac{\tau \text{ fresh} \quad \Gamma[x \mapsto \tau] \vdash t : \tau', S}{\Gamma \vdash \text{fun } x \Rightarrow t : S\tau \rightarrow \tau', S}$$

- 1 Introduce a fresh variable for the argument type, which is still unknown
- 2 Check the body of the function with the argument type added to the context
- 3 Apply the resulting substitution to the argument type

If while typechecking the body, we discover a constraint on the type of the argument, it gets recorded in the substitution

Algorithm W

Step-by-Step

App

$$\frac{\Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \tau' \text{ fresh} \quad S_3 = \text{unify}(S_2 \tau_1, \tau_2 \rightarrow \tau')}{\Gamma \vdash t_1 t_2 : S_3 \tau', S_3 S_2 S_1}$$

- 1 Check the first argument

Algorithm W

Step-by-Step

App

$$\frac{\Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \tau' \text{ fresh} \quad S_3 = \text{unify}(S_2 \tau_1, \tau_2 \rightarrow \tau')}{\Gamma \vdash t_1 t_2 : S_3 \tau', S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument

Algorithm W

Step-by-Step

App

$$\frac{\Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \tau' \text{ fresh} \quad S_3 = \text{unify}(S_2 \tau_1, \tau_2 \rightarrow \tau')}{\Gamma \vdash t_1 t_2 : S_3 \tau', S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Introduce a fresh variable for the return type

Algorithm W

Step-by-Step

App

$$\frac{\Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \tau' \text{ fresh} \quad S_3 = \text{unify}(S_2 \tau_1, \tau_2 \rightarrow \tau')}{\Gamma \vdash t_1 t_2 : S_3 \tau', S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Introduce a fresh variable for the return type
- 4 Update the inferred type of the first argument with new information and try to unify it with the expected function type

Algorithm W

Step-by-Step

App

$$\frac{\Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \tau' \text{ fresh} \quad S_3 = \text{unify}(S_2 \tau_1, \tau_2 \rightarrow \tau')}{\Gamma \vdash t_1 t_2 : \textcolor{red}{S_3} \tau', S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Introduce a fresh variable for the return type
- 4 Update the inferred type of the first argument with new information and try to unify it with the expected function type
- 5 Update the final type with the unification result

Algorithm W

Unification

For two types τ_1 and τ_2 the unification procedure returns the most general substitution S such that $S\tau_1 = S\tau_2$

$$\text{unify}(\text{int}, \text{int}) = \{\}$$

$$\text{unify}(\alpha, \alpha) = \{\}$$

$$\text{unify}(\alpha, \text{int}) = \{\alpha \mapsto \text{int}\}$$

$$\text{unify}(\alpha \rightarrow \beta, \gamma) = \{\gamma \mapsto \alpha \rightarrow \beta\}$$

$$\text{unify}(\alpha \rightarrow \text{int}, \text{bool} \rightarrow \beta) = \{\alpha \mapsto \text{bool}, \beta \mapsto \text{int}\}$$

Algorithm W

Unification Failures

Unification may fail, thus typechecking fails

Algorithm W

Unification Failures

Unification may fail, thus typechecking fails

Incompatible types

$$\text{unify}(\text{int}, \text{bool}) = \perp$$

Algorithm W

Unification Failures

Unification may fail, thus typechecking fails

Incompatible types

$$\text{unify}(\text{int}, \text{bool}) = \perp$$

We cannot unify a variable with a type that contains that variable
It would result in an infinite solution, unsound

$$\text{unify}(\alpha, \text{int} \rightarrow \alpha) = \perp$$

To check for circular occurrences we need to compute the free variables in a type

Algorithm W

Step-by-Step

Plus

$$\frac{S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_3 = \text{unify}(S_2 \tau_1, \text{int}) \quad S_4 = \text{unify}(S_3 \tau_2, \text{int})}{\Gamma \vdash t_1 + t_2 : \text{int}, S_4 S_3 S_2 S_1}$$

- 1 Check the first argument

Algorithm W

Step-by-Step

Plus

$$\frac{S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_3 = \text{unify}(S_2 \tau_1, \text{int}) \quad S_4 = \text{unify}(S_3 \tau_2, \text{int})}{\Gamma \vdash t_1 + t_2 : \text{int}, S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument

Algorithm W

Step-by-Step

Plus

$$\frac{S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_3 = \text{unify}(S_2 \tau_1, \text{int}) \quad S_4 = \text{unify}(S_3 \tau_2, \text{int})}{\Gamma \vdash t_1 + t_2 : \text{int}, S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Update the inferred type of the arguments and unify with `int`

Algorithm W

Step-by-Step

ITE

$$\frac{\begin{array}{c} \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \\ S_2 S_1 \Gamma \vdash t_3 : \tau_3, S_3 \quad S_4 = \text{unify}(S_3 S_2 \tau_1, \text{bool}) \quad S_5 = \text{unify}(S_4 S_3 \tau_2, S_4 \tau_3) \end{array}}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : S_5 S_4 \tau_3, S_5 S_4 S_3 S_2 S_1}$$

- 1 Check the first argument

Algorithm W

Step-by-Step

ITE

$$\frac{\begin{array}{c} \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \\ S_2 S_1 \Gamma \vdash t_3 : \tau_3, S_3 \quad S_4 = \text{unify}(S_3 S_2 \tau_1, \text{bool}) \quad S_5 = \text{unify}(S_4 S_3 \tau_2, S_4 \tau_3) \end{array}}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : S_5 S_4 \tau_3, S_5 S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument

Algorithm W

Step-by-Step

ITE

$$\frac{\begin{array}{c} \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \\ S_2 S_1 \Gamma \vdash t_3 : \tau_3, S_3 \quad S_4 = \text{unify}(S_3 S_2 \tau_1, \text{bool}) \quad S_5 = \text{unify}(S_4 S_3 \tau_2, S_4 \tau_3) \end{array}}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : S_5 S_4 \tau_3, S_5 S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Update the context with the substitution and check the third argument

Algorithm W

Step-by-Step

ITE

$$\frac{S_2 S_1 \Gamma \vdash t_3 : \tau_3, S_3 \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad S_4 = \text{unify}(S_3 S_2 \tau_1, \text{bool}) \quad S_5 = \text{unify}(S_4 S_3 \tau_2, S_4 \tau_3)}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : S_5 S_4 \tau_3, S_5 S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Update the context with the substitution and check the third argument
- 4 Unify the first argument with `bool`

Algorithm W

Step-by-Step

ITE

$$\frac{\begin{array}{c} \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \\ S_2 S_1 \Gamma \vdash t_3 : \tau_3, S_3 \quad S_4 = \text{unify}(S_3 S_2 \tau_1, \text{bool}) \quad S_5 = \text{unify}(S_4 S_3 \tau_2, S_4 \tau_3) \end{array}}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : S_5 S_4 \tau_3, S_5 S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Update the context with the substitution and check the third argument
- 4 Unify the first argument with `bool`
- 5 Unify the second and third arguments

Algorithm W

Step-by-Step

ITE

$$\frac{S_2 S_1 \Gamma \vdash t_3 : \tau_3, S_3 \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma \vdash t_2 : \tau_2, S_2 \quad S_4 = \text{unify}(S_3 S_2 \tau_1, \text{bool}) \quad S_5 = \text{unify}(S_4 S_3 \tau_2, S_4 \tau_3)}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : S_5 S_4 \tau_3, S_5 S_4 S_3 S_2 S_1}$$

- 1 Check the first argument
- 2 Update the context with the substitution and check the second argument
- 3 Update the context with the substitution and check the third argument
- 4 Unify the first argument with `bool`
- 5 Unify the second and third arguments
- 6 Update the return type with the results of unifications

Algorithm W

Step-by-Step

$$\frac{\text{Let} \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma[x \mapsto \text{gener}(S_1 \Gamma, \tau_1)] \vdash t_2 : \tau_2, S_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : \tau_2, S_2 S_1}$$

- 1 Check the term to bind

gener(Γ, τ) takes a monotype and produces a polytype that quantifies over all free variables not in the context, to avoid unwanted capturing

E.g. **gener**($\{\alpha \mapsto \text{int}\}, \alpha \rightarrow \beta \rightarrow \text{bool}$) = $\forall \beta. \alpha \rightarrow \beta \rightarrow \text{bool}$

Algorithm W

Step-by-Step

$$\frac{\text{Let} \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma[x \mapsto \text{gener}(S_1 \Gamma, \tau_1)] \vdash t_2 : \tau_2, S_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : \tau_2, S_2 S_1}$$

- 1 Check the term to bind
- 2 Add the binding to the context, but abstracting over free variables

gener(Γ, τ) takes a monotype and produces a polytype that quantifies over all free variables not in the context, to avoid unwanted capturing

E.g. **gener**($\{\alpha \mapsto \text{int}\}, \alpha \rightarrow \beta \rightarrow \text{bool}$) = $\forall \beta. \alpha \rightarrow \beta \rightarrow \text{bool}$

Algorithm W

Step-by-Step

$$\frac{\text{Let} \quad \Gamma \vdash t_1 : \tau_1, S_1 \quad S_1 \Gamma[x \mapsto \text{gener}(S_1 \Gamma, \tau_1)] \vdash t_2 : \tau_2, S_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : \tau_2, S_2 S_1}$$

- 1 Check the term to bind
- 2 Add the binding to the context, but abstracting over free variables
- 3 Check the body of the let

gener(Γ, τ) takes a monotype and produces a polytype that quantifies over all free variables not in the context, to avoid unwanted capturing

E.g. **gener**($\{\alpha \mapsto \text{int}\}, \alpha \rightarrow \beta \rightarrow \text{bool}$) = $\forall \beta. \alpha \rightarrow \beta \rightarrow \text{bool}$

Algorithm W

Step-by-Step

LetFun

$$\frac{\begin{array}{l} \tau' \text{ fresh} \quad \tau'' \text{ fresh} \quad \Gamma[f \mapsto \tau'][x \mapsto \tau''] \vdash t_1 : \tau_1, S_1 \\ S_2 = \text{unify}(S_1 \tau', S_1 \tau'' \rightarrow \tau_1) \quad S_2 S_1 \Gamma[f \mapsto S_2 S_1 \tau'] \vdash t_2 : \tau_2, S_3 \end{array}}{\Gamma \vdash \text{letfun } f \ x = t_1 \text{ in } t_2 : \tau_2, S_3 S_2 S_1}$$

- 1 Introduce fresh variables for the recursive function and its argument

Algorithm W

Step-by-Step

LetFun

$$\frac{\begin{array}{l} \tau' \text{ fresh} \quad \tau'' \text{ fresh} \quad \Gamma[f \mapsto \tau'][x \mapsto \tau''] \vdash t_1 : \tau_1, S_1 \\ S_2 = \text{unify}(S_1 \tau', S_1 \tau'' \rightarrow \tau_1) \quad S_2 S_1 \Gamma[f \mapsto S_2 S_1 \tau'] \vdash t_2 : \tau_2, S_3 \end{array}}{\Gamma \vdash \text{letfun } f \ x = t_1 \text{ in } t_2 : \tau_2, S_3 S_2 S_1}$$

- 1 Introduce fresh variables for the recursive function and its argument
- 2 Check the body of the function with the extended context

Algorithm W

Step-by-Step

LetFun

$$\frac{\begin{array}{l} \tau' \text{ fresh} \quad \tau'' \text{ fresh} \quad \Gamma[f \mapsto \tau'][x \mapsto \tau''] \vdash t_1 : \tau_1, S_1 \\ S_2 = \text{unify}(S_1 \tau', S_1 \tau'' \rightarrow \tau_1) \quad S_2 S_1 \Gamma[f \mapsto S_2 S_1 \tau'] \vdash t_2 : \tau_2, S_3 \end{array}}{\Gamma \vdash \text{letfun } f \ x = t_1 \text{ in } t_2 : \tau_2, S_3 S_2 S_1}$$

- 1 Introduce fresh variables for the recursive function and its argument
- 2 Check the body of the function with the extended context
- 3 Try to unify the variable for f with its expected function type

Algorithm W

Step-by-Step

LetFun

$$\frac{\begin{array}{l} \tau' \text{ fresh} \quad \tau'' \text{ fresh} \quad \Gamma[f \mapsto \tau'][x \mapsto \tau''] \vdash t_1 : \tau_1, S_1 \\ S_2 = \text{unify}(S_1 \tau', S_1 \tau'' \rightarrow \tau_1) \quad S_2 S_1 \Gamma[f \mapsto S_2 S_1 \tau'] \vdash t_2 : \tau_2, S_3 \end{array}}{\Gamma \vdash \text{letfun } f \ x = t_1 \text{ in } t_2 : \tau_2, S_3 S_2 S_1}$$

- 1 Introduce fresh variables for the recursive function and its argument
- 2 Check the body of the function with the extended context
- 3 Try to unify the variable for f with its expected function type
- 4 Check the body of the let

Algorithm W

Comments

Beside the generation of fresh names, this algorithm does not use side effects

Explicitly applying substitutions make the algorithm clearer and easily implementable also in pure languages

Implementations may use mutation to apply substitution directly, without retraversing types and contexts

Algorithm W

Comments

Beside the generation of fresh names, this algorithm does not use side effects

Explicitly applying substitutions make the algorithm clearer and easily implementable also in pure languages

Implementations may use mutation to apply substitution directly, without retraversing types and contexts

Calls to unify build a system of equation that, when solved, produces the solution

Instead of solving them immediately, they can be accumulated and solved later

This is particularly useful in more complex languages

Project

Fragment 4

- ➊ Define the data structures for monotypes, polytypes and substitutions
- ➋ Implement the functions `inst`, `gener` and `unify`
- ➌ Write a typechecking function for MiniFun, it should have the same interface as the previous fragment

Completing this fragment is not mandatory to pass the exam

It is, however, required to get the maximum grade